

# 01\_EDA Online Retail exploratory data analysis

Anish

2026-05-08

## 1. Project outline

UCI Online Retail (id 352) one year (Dec 2010 – Dec 2011) of transactions from a UK-based online gift retailer.

dataPreprocess.py writes four files into ../data/:

- online\_retail\_clean.csv positive-quantity, positive-price line items
- online\_retail\_cancelled.csv cancellation invoices kept SEPARATELY (so net-of-returns features can be computed without throwing them away)
- customer\_features.csv customer-level RFM and behavioural features
- data\_quality\_summary.txt sanity-check counts

## 2. Load and validate

```
tx      <- read_csv("../data/interim/online_retail_clean.csv",
                    show_col_types = FALSE)
cancelled <- read_csv("../data/interim/online_retail_cancelled.csv",
                    show_col_types = FALSE)
customer  <- read_csv("../data/processed/customer_features.csv",
                    show_col_types = FALSE)

tx$InvoiceDate <- as_datetime(tx$InvoiceDate)
tx$Date <- as_date(tx$Date)
cancelled$InvoiceDate <- as_datetime(cancelled$InvoiceDate)

# Standardise CustomerID (capital ID) and clean Description encoding
# BEFORE any plot or table needs them.
tx <- tx |>
  rename_with(~(n) ifelse(n == "CustomerId", "CustomerID", n)) |>
  mutate(Description = iconv(Description, from = "", to = "UTF-8", sub = ""))

# Validation fail fast if data shape isn't what later chunks assume.
stopifnot(
  "CustomerID" %in% names(tx),
  "CustomerID" %in% names(customer),
  !anyDuplicated(customer$CustomerID),
  nrow(customer) > 0,
```

```

nrow(tx) > 0
)

list(
  transactions = dim(tx),
  cancellations = dim(cancelled),
  customers = dim(customer),
  date_range = range(tx$InvoiceDate),
  countries = n_distinct(tx$Country)
)

## $transactions
## [1] 391150      14
##
## $cancellations
## [1] 8506      9
##
## $customers
## [1] 4334      29
##
## $date_range
## [1] "2010-12-01 08:26:00 UTC" "2011-12-09 12:50:00 UTC"
##
## $countries
## [1] 37

```

```
glimpse(customer)
```

```

## Rows: 4,334
## Columns: 29
## $ CustomerID      <dbl> 12346, 12347, 12348, 12349, 12350, 12352, 12353, 1~
## $ Recency         <dbl> 326, 2, 75, 19, 310, 36, 204, 232, 214, 23, 33, 2, ~
## $ Frequency       <dbl> 1, 7, 4, 1, 1, 7, 1, 1, 1, 3, 1, 2, 4, 3, 1, 10, 2~
## $ GrossMonetary   <dbl> 77183.60, 4310.00, 1437.24, 1457.55, 294.40, 1385.~
## $ TotalQuantity   <dbl> 74215, 2458, 2332, 630, 196, 526, 20, 530, 240, 15~
## $ UniqueProducts   <dbl> 1, 103, 21, 72, 16, 57, 4, 58, 13, 52, 131, 12, 21~
## $ FirstPurchase    <dtm> 2011-01-18 10:01:00, 2010-12-07 14:57:00, 2010-12~
## $ LastPurchase     <dtm> 2011-01-18 10:01:00, 2011-12-07 15:52:00, 2011-09~
## $ TenureDays       <dbl> 0, 365, 282, 0, 0, 260, 0, 0, 0, 302, 0, 149, 274, ~
## $ SinglePurchase   <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, ~
## $ AvgUnitPrice     <dbl> 1.0400000, 1.7534581, 0.6163122, 2.3135714, 1.5020~
## $ AvgOrderValue    <dbl> 77183.6000, 615.7143, 359.3100, 1457.5500, 294.400~
## $ AvgOrderItems    <dbl> 74215.00000, 351.14286, 583.00000, 630.00000, 196.~
## $ ReturnInvoices   <dbl> 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 2, 0, 0, 3, 0, ~
## $ ReturnValueAbs   <dbl> 77183.60, 0.00, 0.00, 0.00, 0.00, 120.33, 0.00, 0.~
## $ Country          <chr> "United Kingdom", "Iceland", "Finland", "Italy", "~
## $ NetMonetary      <dbl> 0.00, 4310.00, 1437.24, 1457.55, 294.40, 1265.41, ~
## $ ReturnRate       <dbl> 0.5000000, 0.0000000, 0.0000000, 0.0000000, 0.0000~
## $ ReturnValueRate  <dbl> 1.000000000, 0.000000000, 0.000000000, 0.000000000~
## $ AvgUnitPrice_winsor <dbl> 1.0400000, 1.7534581, 0.6163122, 2.3135714, 1.5020~
## $ log_Recency      <dbl> 5.789960, 1.098612, 4.330733, 2.995732, 5.739793, ~
## $ log_Frequency    <dbl> 0.6931472, 2.0794415, 1.6094379, 0.6931472, 0.6931~
## $ log_GrossMonetary <dbl> 11.253955, 8.368925, 7.271175, 7.285198, 5.688330, ~

```

```
## $ log_TotalQuantity    <dbl> 11.214735, 7.807510, 7.754910, 6.447306, 5.283204,~
## $ log_UniqueProducts   <dbl> 0.6931472, 4.6443909, 3.0910425, 4.2904594, 2.8332~
## $ log_AvgOrderValue    <dbl> 11.253955, 6.424406, 5.886965, 7.285198, 5.688330,~
## $ log_NetMonetary      <dbl> 0.000000, 8.368925, 7.271175, 7.285198, 5.688330, ~
## $ Monetary             <dbl> 77183.60, 4310.00, 1437.24, 1457.55, 294.40, 1385.~
## $ log_Monetary         <dbl> 11.253955, 8.368925, 7.271175, 7.285198, 5.688330,~
```

### 3. Headline numbers gross AND net

The pipeline now reports both gross sales and net-of-cancellations revenue. The difference is the share of sales customers later cancelled.

```
gross_rev    <- sum(tx$Amount)
return_value <- sum(abs(cancelled$Amount))
net_rev      <- gross_rev - return_value

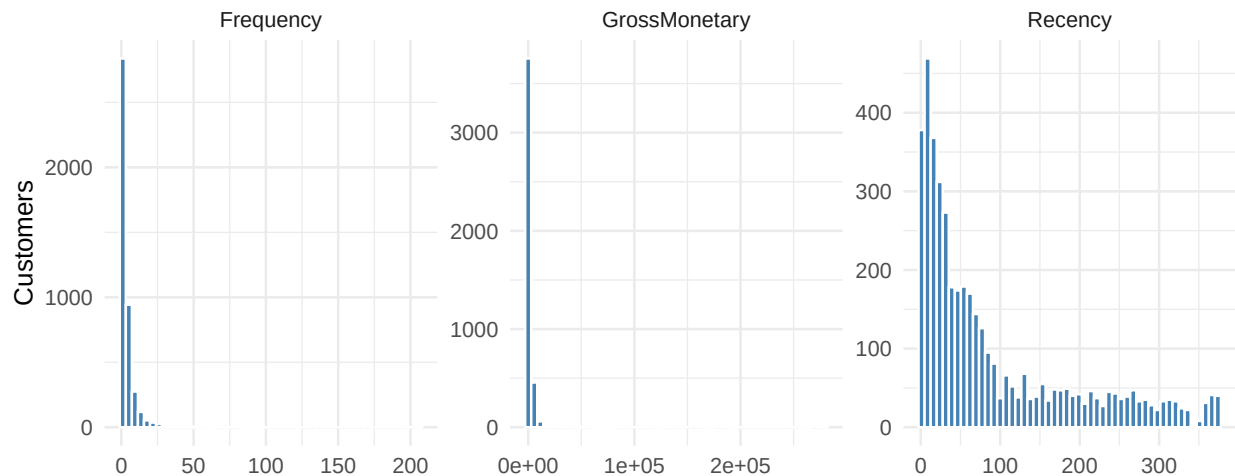
tibble(
  customers      = n_distinct(tx$CustomerID),
  invoices       = n_distinct(tx$InvoiceNo),
  cancelled_invoices = n_distinct(cancelled$InvoiceNo),
  unique_products = n_distinct(tx$StockCode),
  line_items     = nrow(tx),
  gross_revenue  = gross_rev,
  return_value_abs = return_value,
  net_revenue    = net_rev,
  return_value_rate = return_value / gross_rev
) |>
pivot_longer(everything(), names_to = "metric", values_to = "value") |>
mutate(value = case_when(
  str_detect(metric, "rate") ~ percent(value, accuracy = 0.1),
  TRUE                        ~ comma(value, accuracy = 0.01)
))
```

```
## # A tibble: 9 x 2
##   metric      value
##   <chr>      <chr>
## 1 customers  4,334.00
## 2 invoices  18,402.00
## 3 cancelled_invoices 3,383.00
## 4 unique_products  3,659.00
## 5 line_items  391,150.00
## 6 gross_revenue  8,737,227.64
## 7 return_value_abs 471,750.71
## 8 net_revenue    8,265,476.93
## 9 return_value_rate 5.4%
```

### 4. Distribution of RFM features (raw vs. log)

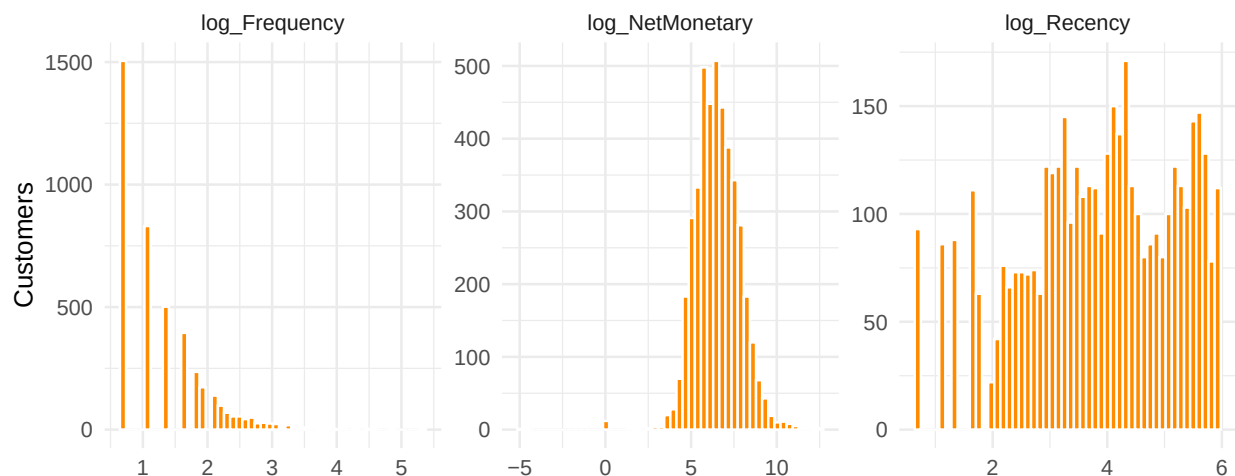
```
customer |>
  select(Recency, Frequency, GrossMonetary) |>
  pivot_longer(everything(), names_to = "feature", values_to = "value") |>
  ggplot(aes(value)) +
  geom_histogram(bins = 50, fill = "steelblue", colour = "white") +
  facet_wrap(~ feature, scales = "free", nrow = 1) +
  labs(title = "Raw RFM distributions (heavy right tail)",
       x = NULL, y = "Customers")
```

Raw RFM distributions (heavy right tail)



```
customer |>
  select(log_Recency, log_Frequency, log_NetMonetary) |>
  pivot_longer(everything(), names_to = "feature", values_to = "value") |>
  ggplot(aes(value)) +
  geom_histogram(bins = 50, fill = "darkorange", colour = "white") +
  facet_wrap(~ feature, scales = "free", nrow = 1) +
  labs(title = "Log-transformed RFM (closer to symmetric, but tail remains)",
       x = NULL, y = "Customers")
```

Log-transformed RFM (closer to symmetric, but tail remains)



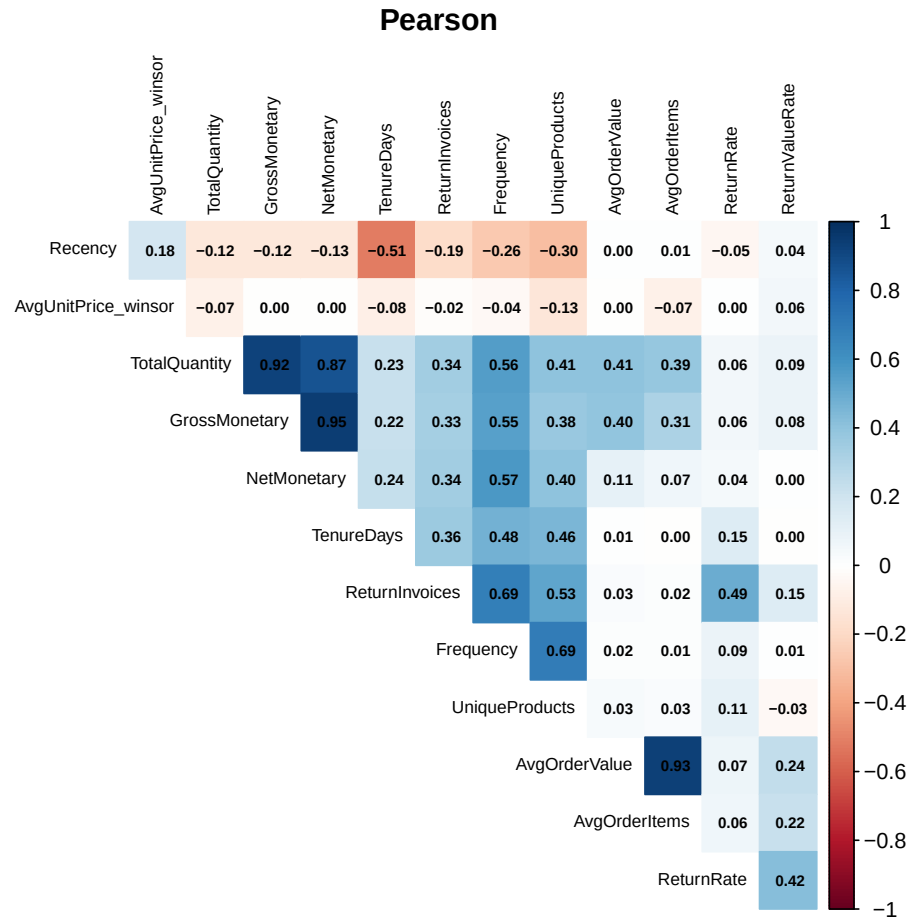
The log transform reduces but does not eliminate skew. `log_NetMonetary` is approximately bell-shaped, but the upper shoulder represents real high-value (likely wholesale-leaning) customers and will continue to influence distance-based clustering.

## 5. Correlation structure Pearson and Spearman

We report both because Pearson is sensitive to outliers; Spearman (rank-based) is robust to them. Pairs that look correlated under Pearson but not Spearman are correlated because of extreme values useful to know when choosing which features to keep.

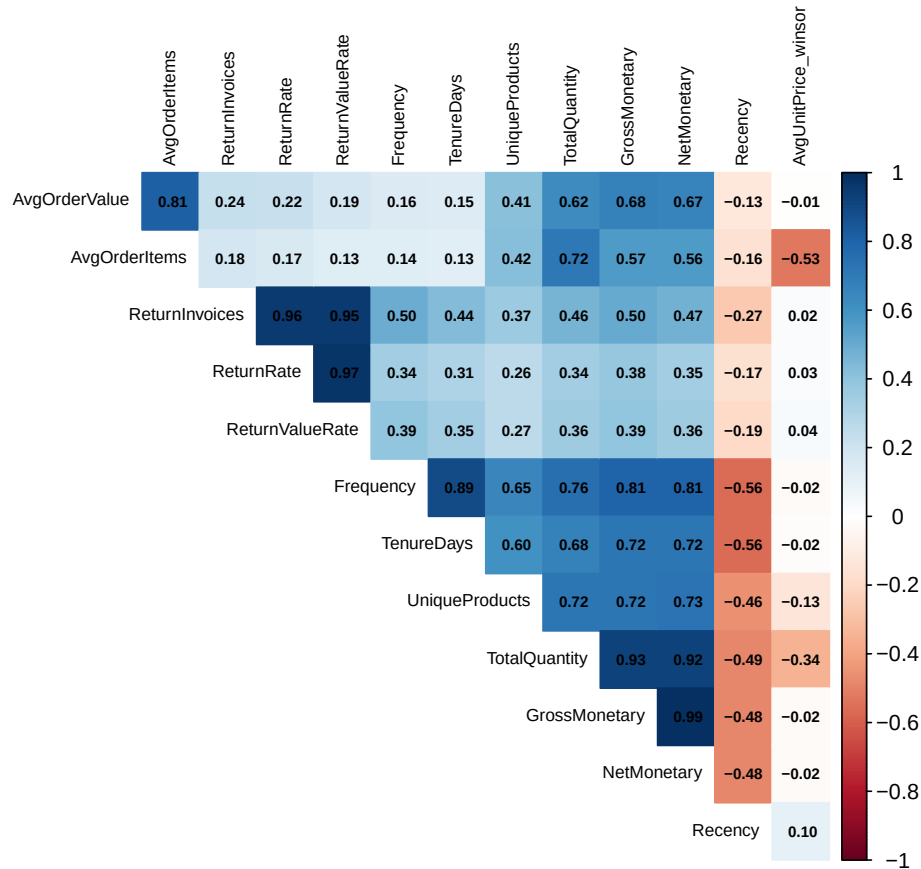
```
num_cols <- customer |>
  select(Recency, Frequency, GrossMonetary, NetMonetary,
         TotalQuantity, UniqueProducts,
         AvgUnitPrice_winsor, AvgOrderValue, AvgOrderItems,
         TenureDays, ReturnInvoices, ReturnRate, ReturnValueRate)

corrplot::corrplot(
  cor(num_cols, use = "pairwise.complete.obs"),
  method = "color", type = "upper", order = "hclust",
  addCoef.col = "black", number.cex = 0.6,
  tl.col = "black", tl.cex = 0.7, diag = FALSE,
  title = "Pearson", mar = c(0, 0, 2, 0)
)
```



```
corrplot::corrplot(
  cor(num_cols, use = "pairwise.complete.obs", method = "spearman"),
  method = "color", type = "upper", order = "hclust",
  addCoef.col = "black", number.cex = 0.6,
  tl.col = "black", tl.cex = 0.7, diag = FALSE,
  title = "Spearman (rank-based)", mar = c(0, 0, 2, 0)
)
```

## Spearman (rank-based)



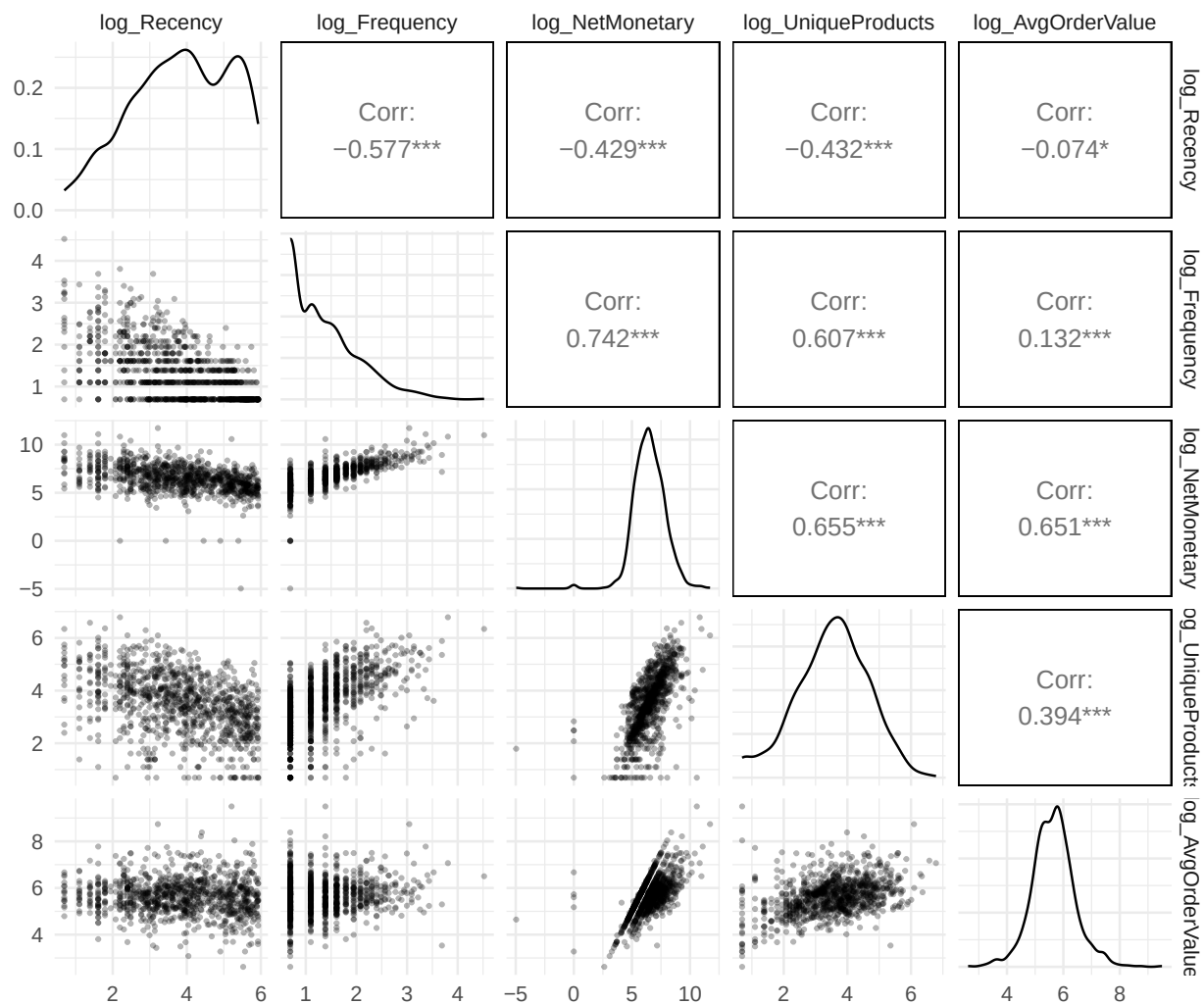
What this implies for clustering input selection:

- **GrossMonetary**, **NetMonetary**, **TotalQuantity**, and **Frequency** form a tight collinear block. We keep **NetMonetary** because it captures the customer's *actual* contribution to revenue net of returns.
- **AvgOrderValue** and **AvgOrderItems** carry similar information; we keep **AvgOrderValue** (revenue-relevant) and drop **AvgOrderItems**.
- **Recency** is largely independent of the spending features under both correlation measures it is its own axis of variation, justifying the canonical R, F, M triple.
- **ReturnRate** and **ReturnValueRate** are weakly correlated with everything else. They are best used as *profiling* variables, not clustering inputs.

## 6. Pairs plot on the modelling set

```
set.seed(1)
customer |>
  slice_sample(n = 1000) |>
  select(log_Recency, log_Frequency, log_NetMonetary,
         log_UniqueProducts, log_AvgOrderValue) |>
  GGally::ggpairs(progress = FALSE,
                  lower = list(continuous = wrap("points", alpha = 0.3,
                                                  size = 0.6))) +
  labs(title = "Log-feature pairs plot (1k-customer sample)")
```

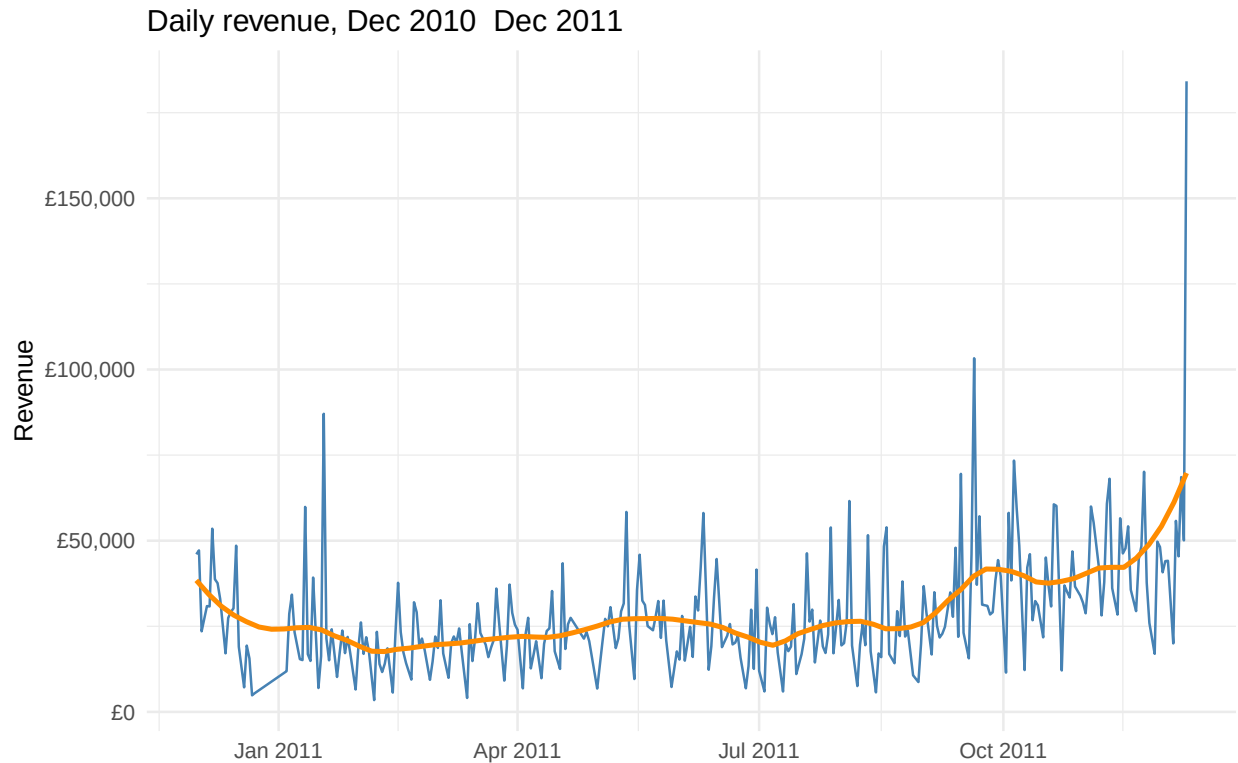
Log-feature pairs plot (1k-customer sample)



## 7. Sales over time

```
daily <- tx |>
  group_by(Date) |>
  summarise(revenue = sum(Amount),
            invoices = n_distinct(InvoiceNo), .groups = "drop")

ggplot(daily, aes(Date, revenue)) +
  geom_line(colour = "steelblue") +
  geom_smooth(method = "loess", span = 0.2, se = FALSE,
             colour = "darkorange") +
  scale_y_continuous(labels = label_comma(prefix = pound)) +
  labs(title = paste("Daily revenue, Dec 2010", emdash, "Dec 2011"),
       x = NULL, y = "Revenue")
```



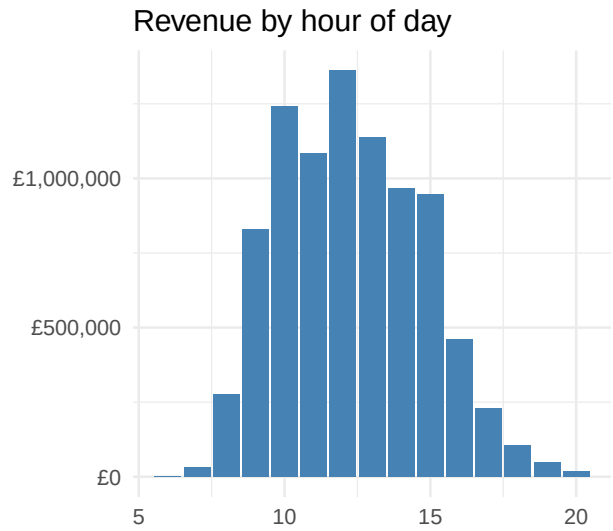
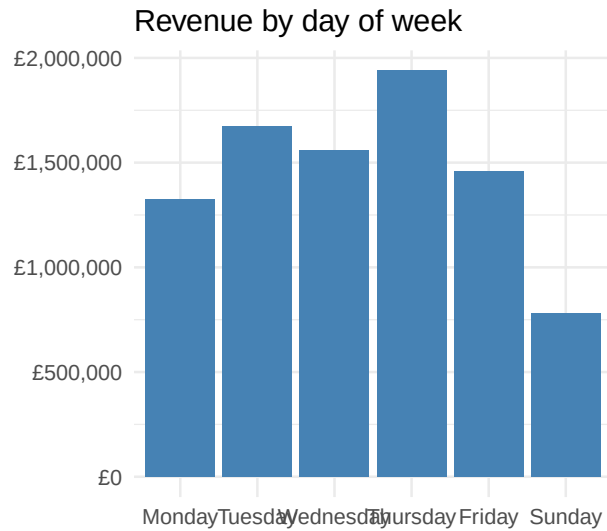
The Q4 2011 ramp is the holiday gift season a sanity check that the timestamps and currency conversions are correct. The single-day spikes in late November / early December correspond to a small number of very large invoices.

```
library(gridExtra)

p_dow <- tx |>
  dplyr::count(DOW = factor(DOW,
    levels = c("Monday", "Tuesday", "Wednesday", "Thursday",
      "Friday", "Saturday", "Sunday")),
    wt = Amount, name = "revenue") |>
  ggplot(aes(DOW, revenue)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = label_comma(prefix = pound)) +
  labs(title = "Revenue by day of week", x = NULL, y = NULL)

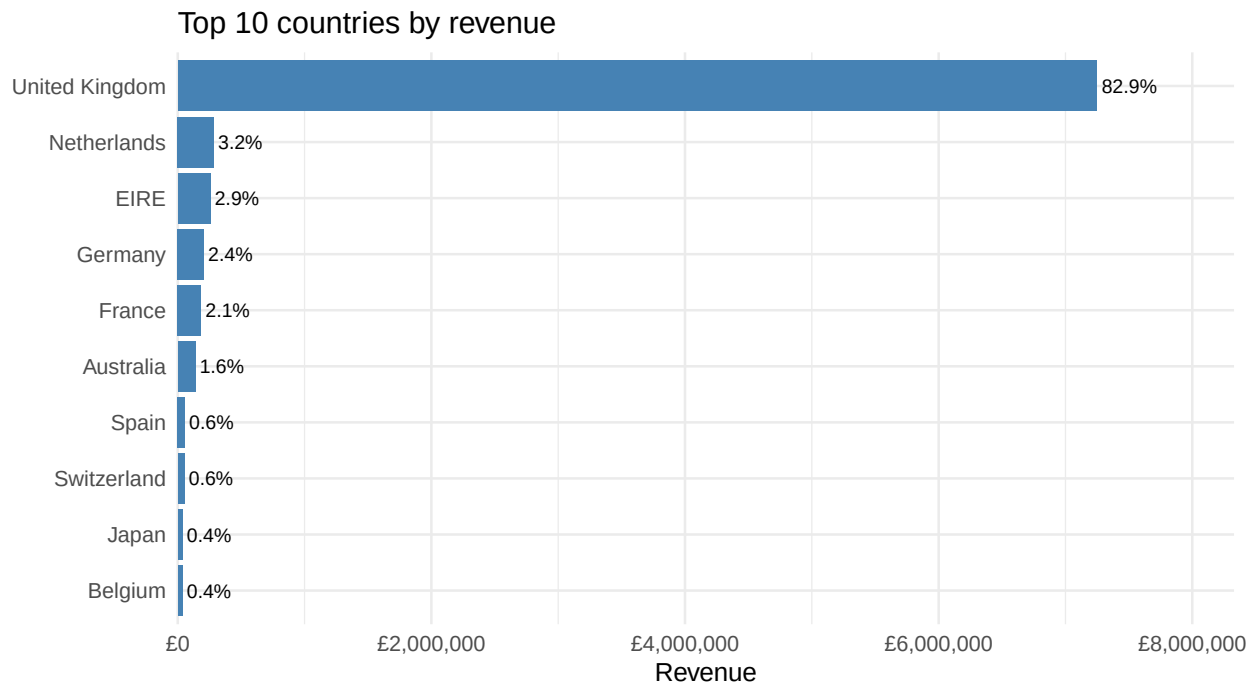
p_hour <- tx |>
  dplyr::count(Hour, wt = Amount, name = "revenue") |>
  ggplot(aes(Hour, revenue)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = label_comma(prefix = pound)) +
  labs(title = "Revenue by hour of day", x = NULL, y = NULL)

gridExtra::grid.arrange(p_dow, p_hour, nrow = 1)
```



## 8. Geography

```
tx |>
  dplyr::count(Country, wt = Amount, name = "revenue", sort = TRUE) |>
  mutate(
    share = revenue / sum(revenue),
    Country = fct_reorder(Country, revenue)
  ) |>
  slice_max(revenue, n = 10) |>
  ggplot(aes(revenue, Country)) +
  geom_col(fill = "steelblue") +
  geom_text(
    aes(label = percent(share, accuracy = 0.1)),
    hjust = -0.1,
    size = 3
  ) +
  scale_x_continuous(
    labels = label_comma(prefix = pound),
    expand = expansion(mult = c(0, 0.15))
  ) +
  labs(
    title = "Top 10 countries by revenue",
    x = "Revenue",
    y = NULL
  ) +
  coord_cartesian(clip = "off")
```



The UK dominates roughly four-fifths of net revenue, which is why `Country` is held out of the clustering input matrix in `O2_PCA.Rmd`: including it would force a near-degenerate UK / non-UK split that adds no behavioural signal.

## 9. Anomaly check single-day revenue spikes

The smoothed loess in §7 shows real seasonality, but the residuals reveal a handful of days whose revenue is several standard deviations above the local trend. Distance-based clusterers are sensitive to a small number of extreme observations, so we have to know which invoices and which products drive these spikes before deciding what if anything to do with them.

### 9a. Days with the largest residual revenue

```
daily_res <- daily |>
  arrange(Date) |>
  mutate(trend = predict(loess(revenue ~ as.numeric(Date),
                              data = pick(everything()),
                              span = 0.2)),
         resid = revenue - trend) |>
  arrange(desc(resid))

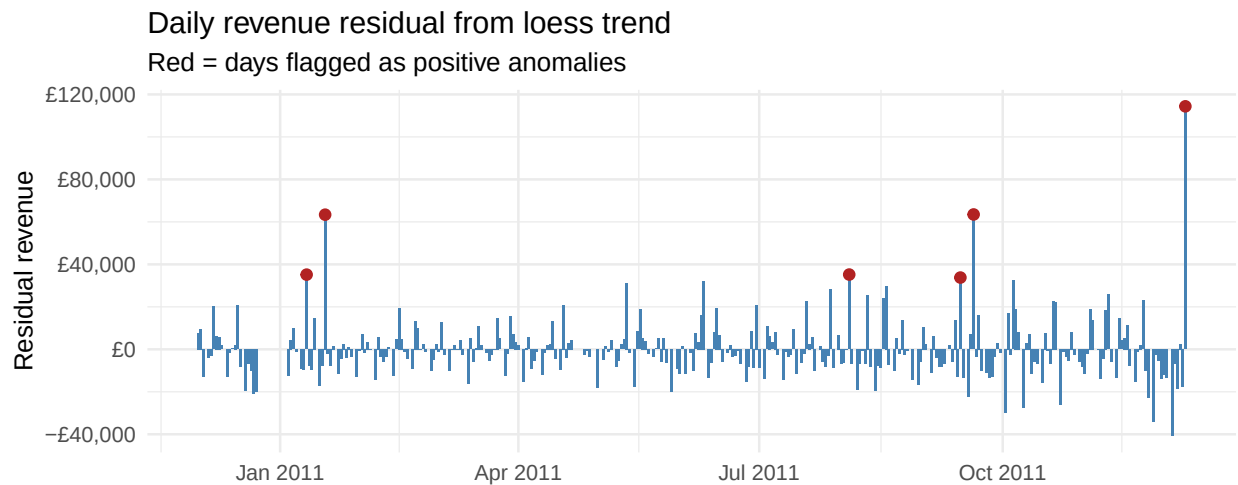
top_days <- daily_res |> slice_head(n = 6)

top_days |>
  mutate(across(c(revenue, trend, resid),
                 \(x) comma(x, accuracy = 1, prefix = pound))) |>
  knitr::kable(caption = "Six days with the largest positive residual revenue")
```

Table 1: Six days with the largest positive residual revenue

| Date       | revenue  | invoices | trend   | resid    |
|------------|----------|----------|---------|----------|
| 2011-12-09 | £184,171 | 41       | £69,764 | £114,407 |
| 2011-09-20 | £103,280 | 67       | £39,790 | £63,490  |
| 2011-01-18 | £87,066  | 31       | £23,679 | £63,387  |
| 2011-08-04 | £61,600  | 79       | £26,397 | £35,203  |
| 2011-01-11 | £59,865  | 53       | £24,700 | £35,165  |
| 2011-09-15 | £69,521  | 76       | £35,732 | £33,788  |

```
ggplot(daily_res, aes(Date, resid)) +
  geom_col(fill = "steelblue") +
  geom_point(data = top_days, aes(Date, resid),
             colour = "firebrick", size = 2) +
  scale_y_continuous(labels = label_comma(prefix = pound)) +
  labs(title = "Daily revenue residual from loess trend",
       subtitle = "Red = days flagged as positive anomalies",
       x = NULL, y = "Residual revenue")
```



## 9b. Top contributing invoices on those days

```
top_invoices <- tx |>
  filter(Date %in% top_days$Date) |>
  group_by(Date, InvoiceNo, CustomerID, Country) |>
  summarise(invoice_revenue = sum(Amount),
            line_items      = n(),
            .groups = "drop") |>
  arrange(desc(invoice_revenue)) |>
  slice_head(n = 10)

top_invoices |>
  mutate(invoice_revenue = comma(invoice_revenue, accuracy = 1,
                                prefix = pound)) |>
  knitr::kable(caption = "Top 10 invoices on flagged days")
```

Table 2: Top 10 invoices on flagged days

| Date       | InvoiceNo | CustomerID | Country        | invoice_revenue | line_items |
|------------|-----------|------------|----------------|-----------------|------------|
| 2011-12-09 | 581483    | 16446      | United Kingdom | £168,470        | 1          |
| 2011-01-18 | 541431    | 12346      | United Kingdom | £77,184         | 1          |
| 2011-09-20 | 567423    | 17450      | United Kingdom | £31,698         | 12         |
| 2011-09-20 | 567381    | 17450      | United Kingdom | £22,105         | 12         |
| 2011-08-04 | 562439    | 12931      | United Kingdom | £18,841         | 22         |
| 2011-01-11 | 540815    | 15749      | United Kingdom | £15,161         | 3          |
| 2011-09-15 | 566934    | 18102      | United Kingdom | £13,250         | 8          |
| 2011-01-11 | 540689    | 17450      | United Kingdom | £12,798         | 4          |
| 2011-01-11 | 540818    | 15749      | United Kingdom | £7,838          | 2          |
| 2011-09-15 | 566935    | 18102      | United Kingdom | £6,870          | 5          |

### 9c. Top contributing products

```

top_products <- tx |>
  filter(Date %in% top_days$Date) |>
  group_by(StockCode, Description) |>
  summarise(revenue = sum(Amount),
            quantity = sum(Quantity),
            invoices = n_distinct(InvoiceNo),
            .groups = "drop") |>
  arrange(desc(revenue)) |>
  slice_head(n = 8)

top_products |>
  mutate(Description = str_trunc(Description, 35),
         revenue = comma(revenue, accuracy = 1, prefix = pound),
         quantity = comma(quantity, accuracy = 1)) |>
  knitr::kable(caption = "Top 8 products on flagged days")

```

Table 3: Top 8 products on flagged days

| StockCode | Description                            | revenue  | quantity | invoices |
|-----------|----------------------------------------|----------|----------|----------|
| 23843     | PAPER CRAFT , LITTLE BIRDIE            | £168,470 | 80,995   | 1        |
| 23166     | MEDIUM CERAMIC TOP STORAGE JAR         | £77,299  | 74,315   | 8        |
| 85123A    | WHITE HANGING HEART T-LIGHT<br>HOLDER  | £10,691  | 3,800    | 38       |
| 23243     | SET OF TEA COFFEE SUGAR TINS<br>PANTRY | £7,651   | 1,526    | 13       |
| 48185     | DOORMAT FAIRY CAKE                     | £6,936   | 1,194    | 4        |
| 21108     | FAIRY CAKE FLANNEL ASSORTED<br>COLOUR  | £6,637   | 3,177    | 6        |
| 22470     | HEART OF WICKER LARGE                  | £6,346   | 1,989    | 18       |
| 84879     | ASSORTED COLOUR BIRD ORNAMENT          | £5,997   | 4,071    | 26       |

The most consistent contributor across the flagged days is **StockCode 23843** (“PAPER CRAFT, LITTLE BIRDIE”), bought on a single very large wholesale invoice. Two implications:

1. **No removal.** The line is a legitimate transaction by a real customer. Dropping it would silently bias the segment definitions and break reproducibility against the published UCI table.
2. **Profile with and without.** `04_profiles.Rmd` reports the top products per cluster *both* including and excluding 23843 so that one customer's single bulk order does not overstate the assortment of the segment they sit in.